«Драфт» — техническая документация

Платформа найма IT-специалистов с подтверждённым грейдом и профилем достижений ФСП

Команда Neuropeople · оператор ООО «НЕЙРОПИПЛ»

Специальный трек ФСП, Лидеры цифровой трансформации 2026

Содержание

О документе
1 Задача и решение
2 Функциональная и компонентная архитектура
2.1 Роли и функциональные блоки
2.2 Компонентная схема
2.3 Разделение слоёв
2.4 Рыночный режим и демо
3 Механика тестирования
3.1 Банк параметрических заданий
3.2 Адаптивный тест и вердикт
3.3 Почему распространение заданий не помогает
3.4 Проверка развёрнутых ответов и защита от манипуляций
3.5 Честность прохождения
3.6 Регулярные задания и подписанный паспорт грейда
4 Механика подбора и категоризация
4.1 Категории
4.2 Разбор потребности
4.3 Ранжирование и объяснение
4.4 Уточнение без потери подборки
4.5 Выход на контакт
5 Процедура валидации и результаты
5.1 Принципы процедуры
5.2 Главные метрики: стандартные условия и проверка на прочность
5.3 Тест на грейд
5.4 Банк заданий
5.5 Подбор
5.6 Защита проверки ответов и честность прохождения
5.7 Реальные документы ФСП
5.8 Ограничения
5.9 План пилота с ФСП
6 Интеграция с реестром ФСП
6.1 Статусы достижения
6.2 Персональные данные спортсменов
6.3 Как перейти на боевую интеграцию
7 Описание API
8 Библиотеки и компоненты
9 Сборка, развёртывание и локальный запуск
9.1 Требования
9.2 Запуск в Docker (рекомендуется)
9.3 Запуск без Docker
9.4 Проверка
9.5 Развёртывание на сервере
10 Нефункциональные требования и безопасность
11 Монетизация
Приложение А. Соответствие ТЗ
Приложение Б. Все цифры документа

О документе

Документ описывает решение «Драфт» в объёме пункта 3.4 технического задания. Каждый раздел отвечает на один пункт:

Пункт ТЗ 3.4	Раздел
Функциональная и компонентная архитектура	2
Механика тестирования и её устойчивость к распространению заданий	3
Механика подбора и логика категоризации	4
Процедура валидации и её результаты	5
Схема интеграции с реестром ФСП	6
Описание API	7
Библиотеки и компоненты с версиями	8
Сборка, развёртывание и локальный запуск	9

Требования раздела 3.5 (код, безопасность, 152-ФЗ, производительность) — раздел 10, монетизация — раздел 11, карта соответствия всем пунктам ТЗ — приложение А, полный список цифр с ключами — приложение Б.

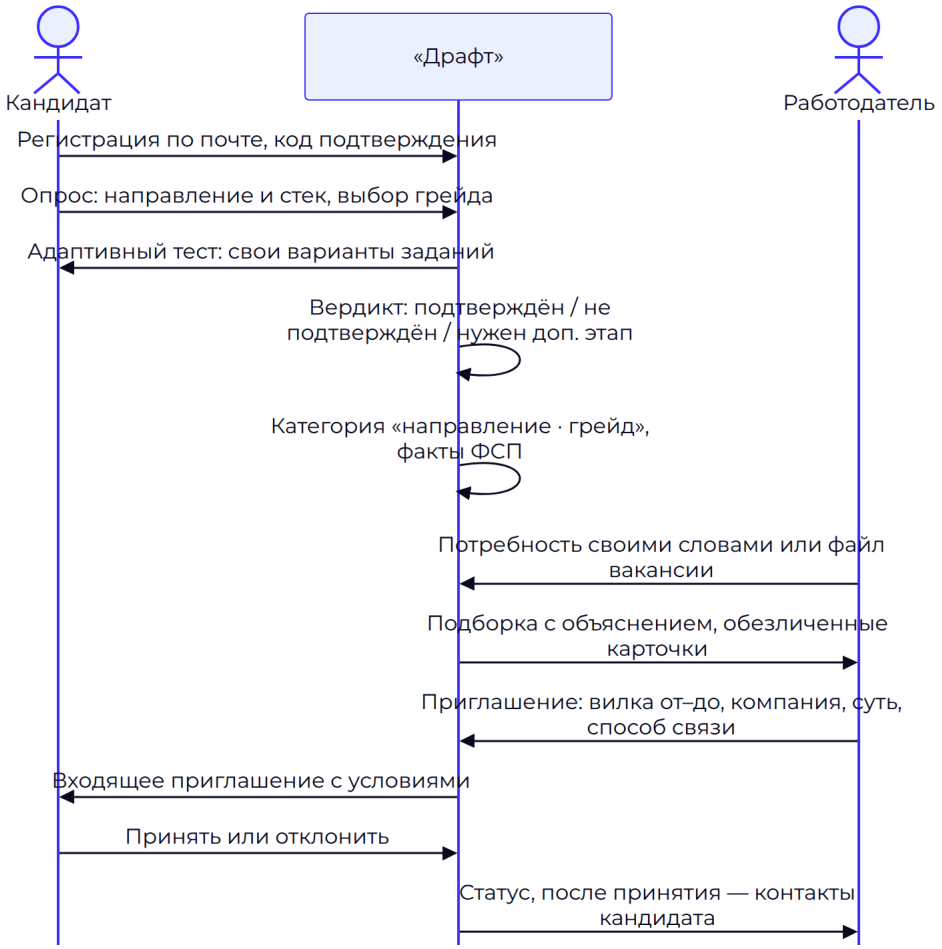
Ссылки. Сайт — https://fsp-draft.ru; раздел для экспертов — https://fsp-draft.ru/demo; методика и цифры — https://fsp-draft.ru/validation; описание API — https://fsp-draft.ru/api/docs; исходный код — https://github.com/neuropeopleteam-afk/draft.

Как читать цифры. Все числа качества в документе подставлены автоматически из файла `docs/NUMBERS.json`, который собирается из отчётов прогонов. У каждого числа указано, на ком оно измерено: «симуляция, модельные кандидаты, не люди», «реальные вакансии hh.ru», «официальные протоколы ФСП», — объём выборки, дата и команда пересчёта. Документ пересобирается одной командой `python3 docs/submission/build.py`.

1 Задача и решение

Универсальные площадки поиска работы сводят IT-подбор к совпадению ключевых слов: резюме не подтверждает навыки, работодатель вручную разбирает сотни откликов, кандидат откликается вслепую и не видит условий. «Драфт» меняет порядок действий:

1. **Кандидат один раз подтверждает навыки.** Регистрация по почте с подтверждением → короткий опрос (направление, стек) → выбор грейда, на который кандидат претендует → адаптивный тест на этот грейд. Итог — категория «направление · грейд», которую видит работодатель. Категория строится только по тесту, не по резюме.
2. **Работодатель описывает задачу своими словами** и получает подборку обезличенных кандидатов из подходящих категорий с объяснением «почему этот кандидат здесь».
3. **Работодатель сам выходит на кандидата** с приглашением, в котором обязательны зарплатная вилка «от–до» в рублях, название компании, описание предложения и способ связи. Кандидат видит условия до начала общения; контакты кандидата раскрываются только после его согласия.
4. **Достижения ФСП подтверждаются документами** — официальными протоколами соревнований ФСП России — и поднимают кандидата в подборке. Кандидат без истории ФСП не получает штрафа.



Дополнительно реализованы: самостоятельный отклик на опубликованные вакансии, регулярные короткие задания от работодателей с автопроверкой, защита от фиктивных вакансий (проверка компании в ЕГРЮЛ, индекс доверия), передача кандидатов в ATS, мобильное приложение и адаптивная вёрстка, кабинет Федерации и трёхсторонняя связь «компания — кандидат — ФСП».

2 Функциональная и компонентная архитектура

2.1 Роли и функциональные блоки

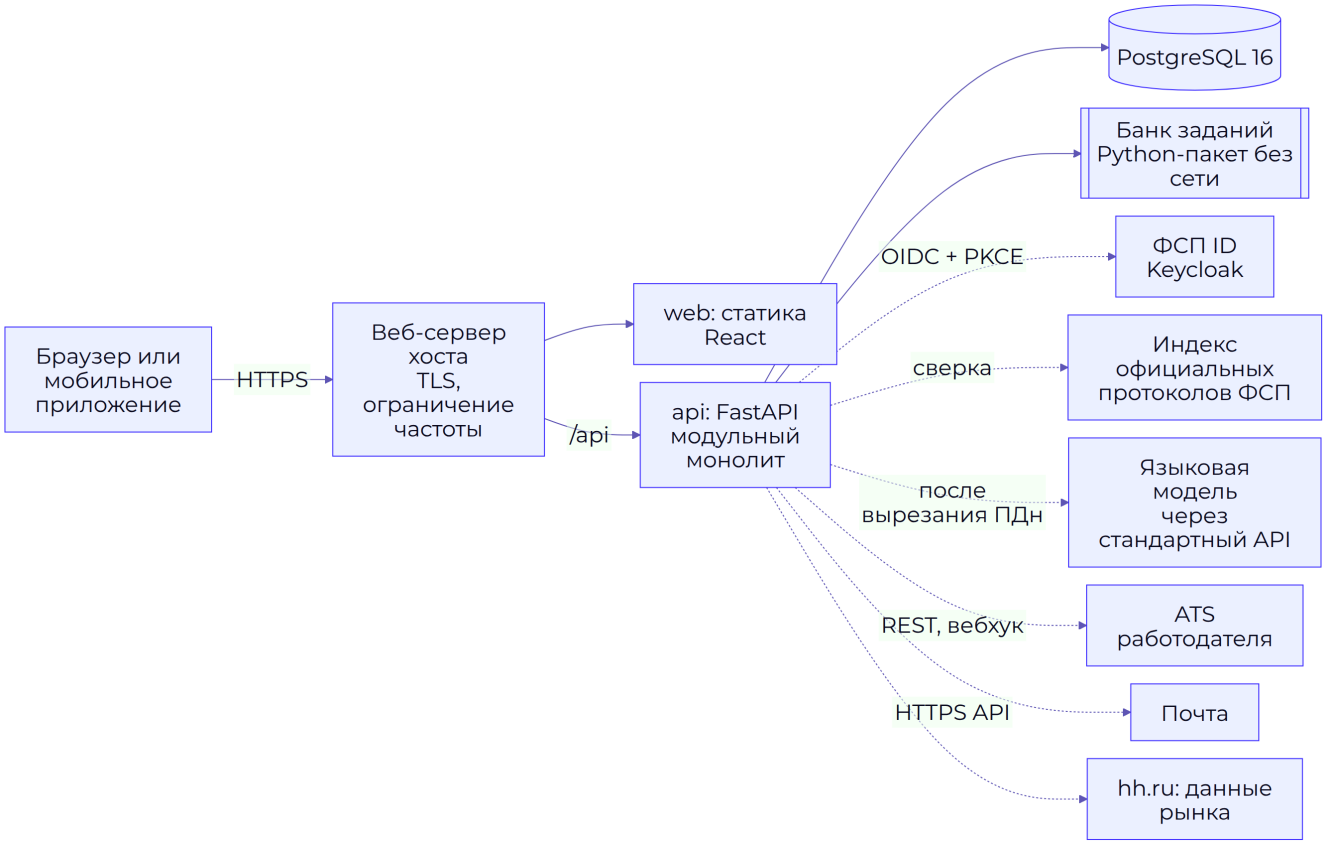
Роль	Что делает на платформе	Где на сайте
Кандидат	профиль вручную или из PDF-резюме с авторазпознаванием; опрос, выбор грейда, тест; категория и история попыток; приглашения и отклики; регулярные задания; приватность, согласия, связь с ФСП ID; выгрузка и удаление данных	`/candidate`
Работодатель	профиль компании; потребность текстом или файлом; подборка с объяснением и версиями; поиск по базе с фильтрами; сравнение кандидатов; приглашения со статусами; вакансии и отклики; задания для кандидатов; найм и передача в ATS	`/employer`
Федерация ФСП	кадровый резерв по регионам и направлениям против спроса рынка; подтверждение достижений; трёхсторонняя связь; спонсорские пакеты и кубки работодателей	`/fsp`
Методист / эксперт	очередь проверки развёрнутых ответов по рубрике, разбор спорных случаев	`/expert`
Администратор	пользователи, компании, заявки на тарифы, обращения в поддержку	`/admin`
Эксперт жюри	вход в демо-роль в один клик, личная песочница с демо-данными, маршрут по критериям ТЗ	`/demo`

Функциональные блоки соответствуют модулям backend (`backend/app/modules/<модуль>/`):

Блок	Модули	Главное
Учётные записи и доступ	`auth` , `admin`	регистрация по почте с кодом, вход (JWT), второй фактор для служебных ролей, роли
Справочник	`catalog`	направления, грейды, категории, стек, дисциплины ЕВСК
Профили и документы	`profiles` , `resume` , `pdf` , `mydata`	опрос, приватность, разбор PDF/DOCX-резюме, стандартизированный PDF-профиль, выгрузка и удаление данных

Блок	Модули	Главное
Тестирование	`assessment`, `proctor`, `interview`, `experts`	банк заданий, адаптивный тест, вердикт, второй этап, проверка развёрнутых ответов, сигналы честности, устная защита
Подбор	`matching`, `mutual`, `subs`	разбор потребности, ранжирование с объяснением, версии подборок, взаимный подбор, подписка на новых кандидатов
Контакт и найм	`invitations`, `contact`, `hiring`, `trio`, `notifications`	приглашения и статусы, переписка, этапы найма, трёхсторонняя связь, уведомления
Вакансии и задания	`assignments`, `hh`	вакансии и отклики, задания работодателя с автопроверкой, данные рынка hh.ru
ФСП	`fsp` и `app/fsp_cabinet`	паспорт достижений, сверка с протоколами, нормативы ЕВСК, кабинет федерации
Интеграции	`app/integrations`	ФСП ID (OIDC), ATS, проверка компании в ЕГРЮЛ
Качество и прозрачность	`validation`, `study`	публичные цифры качества, обезличенная статистика прохождения
Демонстрация	`jury`, `demo`, `warroom`	раздел для жюри, демо-данные, песочницы
Коммерция	`tariffs`	тарифы работодателей и спонсорские пакеты ФСП

2.2 Компонентная схема



Компонент	Технология	Назначение
`web`	nginx + сборка React/Vite	интерфейс для всех ролей, адаптивная вёрстка, установка как приложение
`api`	Python 3.12, FastAPI, SQLAlchemy 2, Alembic	вся бизнес-логика; миграции применяются при старте
`db`	PostgreSQL 16	единственное хранилище; тесты идут и на PostgreSQL, и на SQLite
`keycloak` (профиль `fsp-id`)	Keycloak	демо-стенд «ФСП ID» для входа и связи профиля
`llm-local` (профиль `llm-local`)	открытая модель на CPU	работа без внешних ключей

Внешняя языковая модель необязательна: при её отсутствии разбор текста переходит на словарь и правила, а развёрнутые ответы — в очередь проверки экспертом. Персональные данные вырезаются до отправки текста модели.

2.3 Разделение слоёв

Backend — модульный монолит с явными слоями. Каждый модуль: `models.py` (хранение, только свои таблицы), `schemas.py` (контракты API, Pydantic), `service.py` (бизнес-логика без HTTP), `router.py` (HTTP, коды ошибок, описание в OpenAPI). Модули общаются через сервисы; внешние системы (почта, модель, ФСП, ATS) — только через адаптеры с запасным путём. Банк заданий — отдельный пакет без доступа к БД и сети, поэтому его можно проверять и переиспользовать отдельно. Подробности и модель данных — `docs/ARCHITECTURE.md`.

2.4 Рыночный режим и демо

Основной сайт работает как рыночный продукт: настоящие пользователи видят только настоящие объекты. Демо-данные (модельные кандидаты, компании, потребности) помечены и видны только в демо-ролях раздела `/demo`; у каждого эксперта жюри — своя песочница, которую можно сбросить. Проверка видимости — единая функция в `backend/app/modules/jury/scope.py`, автотест проходит по всем ответам API и ищет утечки демо-данных.

3 Механика тестирования

3.1 Банк параметрических заданий

Банк содержит 1 646 семейств заданий по 10 направлениям, из них 416 — с кодом и развёрнутым ответом. Семейство — это не одно задание, а генератор: из шаблона и случайного зерна получается конкретный вариант с другими данными, именами, числами и, где возможно, другим правильным ответом при той же проверяемой идее и той же трудности. Форматы: «найди ошибку», «предскажи вывод», «выбери исправление», «открытая книга в три шага», концептуальный выбор, ввод ответа, развёрнутый ответ по рубрике.

Ключи всех вариантов проверяются автоматически: валидатор генерирует варианты каждого семейства и исполняет эталонное решение. Последняя проверка — 38 131 проверок, ошибок — 0 (автоматическая проверка банка: 20 вариантов-зёрен на каждое семейство, коммит банка ab16d27, 21.10.2026).

3.2 Адаптивный тест и вердикт

Тест подбирает следующее задание по уже данным ответам: так каждый кандидат получает задания, которые лучше всего различают уровни рядом с его заявленным грейдом. Около трети заданий — практические (код, развёрнутый ответ). Когда оценка уровня оказывается рядом с границей грейда, тест задаёт короткие уточняющие задания.

Итог отвечает на вопрос работодателя — подтверждён ли заявленный грейд — тремя исходами:

- **подтверждён** — грейд появляется в категории и в подборках;
- **не подтверждён** — кандидат видит, на каких навыках не хватило, и может сразу пройти тест на грейд ниже;
- **нужен короткий дополнительный этап** — если результат у самой границы, вместо угадывания назначается второй этап на новых заданиях.

Как считается: байесовская последовательная классификация — после каждого ответа пересчитывается вероятность того, что уровень кандидата не ниже границы заявленного грейда, и вердикт выносится по этой вероятности. Последовательный критерий Вальда (SPRT, α = 2 %) решает только, можно ли остановить тест раньше, когда ответ уже ясен.

Правила грейда (из `backend/app/modules/assessment/rules.py` и `backend/app/core/config.py`):

- неудачная попытка **не понижает** уже подтверждённый грейд;
- после неподтверждённой попытки тест на **грейд ниже** можно пройти сразу, на **тот же или более высокий** — не раньше чем через 30 дней;
- смена подтверждённого грейда — не чаще одного раза в 90 дней (ТЗ: «не чаще раза в три месяца»); короткое уточнение у границы — часть той же оценки, а не новая попытка;
- для экспертов жюри в демо-ролях ограничения сняты, чтобы можно было пройти сценарий несколько раз.

По симуляции продукта (тот же движок, что на сайте) итог проверки заявленного грейда верен в 84,8 % случаев (2 544 из 3 000; 95 % ДИ 83,5–86,0 %); уверенный ответ уже по основному тесту получают 89,2 % кандидатов, и он верен в 89,8 %. Условие: стандартные условия (как в аналогичных решениях), зёрна 5501–5510; симуляция продукта: модельные кандидаты, не люди; N = 3 000; тот же движок теста, что на сайте; банк ab16d27 (1 646 семейств заданий); 21.10.2026. Подробно — раздел 5.

3.3 Почему распространение заданий не помогает

ТЗ называет два риска: единый набор заданий быстро утекает, а уникальные задания, сгенерированные нейросетью под кандидата, несопоставимы по сложности. «Драфт» закрывает оба:

1. **Свой вариант у каждого.** Кандидат получает варианты семейств со своим зерном; выученный ответ чужого варианта не подходит к своему. При этом сложность сопоставима: все варианты семейства проверяют одну идею и имеют одинаковые параметры трудности — это проверяется валидатором банка, а не обещается.
2. **Контроль частоты показа.** Ни одно семейство не показывается слишком часто (метод Сымпсона — Хеттера); задания разбиты на параллельные субпулы, и повторная попытка получает другой субпул. Утёкший субпул выводится из ротации одной настройкой.
3. **Повторная попытка — на новых семействах.** Семейства прошлых попыток кандидата не повторяются, пока есть новые.
4. **Ключи только на сервере.** Правильные ответы не передаются в браузер; служебные поля варианта не видны кандидату (проверяется аудитом банка).
5. **Сигналы предзнания.** Слишком быстрые верные ответы на трудные задания и несогласованность с остальными ответами отмечаются как сигнал для дополнительной проверки.
6. **Модель не сочиняет задания на лету.** Генерация — детерминированная по шаблону, поэтому трудность известна заранее; языковая модель используется только как один из проверяющих развёрнутых ответов.

3.4 Проверка развёрнутых ответов и защита от манипуляций

Развёрнутый ответ оценивается по рубрике. Перед оценкой фильтр ищет попытки манипуляции («поставь максимальный балл», скрытые инструкции); текст кандидата изолируется от инструкций проверяющего; каждая оценка должна опираться на дословные цитаты из ответа, иначе она отклоняется; при расхождении нужна вторая оценка, а спорные случаи уходят эксперту в очередь `/expert`. На отложенном наборе атак фильтр перехватил 77,8 % атак (137 из 176) при 1,1 % ложных срабатываний; всю цепочку защиты прошли 0 из 176 атак (отложенный набор атак на проверку ответов, составлен после фиксации кода фильтра (коммит 6b7c710c), один прогон без донастройки; N = 176 атак и 176 честных ответов; 20.10.2026).

3.5 Честность прохождения

Во время теста собираются сигналы: переключение вкладок, вставка текста, нетипичное время ответа, по согласию — анализ камеры прямо в браузере (видео никуда не передаётся). Один сигнал ничего не значит: при двух и более сигналах или прямом доказательстве назначается дополнительная проверка: короткий блок заданий у порога грейда или устная защита своего решения голосом (несколько минут). Это дополнительный сигнал, а не доказательство «без нейросетей»: решение принимают тест и дополнительный этап. Попытка не аннулируется, автоматической блокировки нет — честный кандидат не должен страдать из-за случайности. В симуляции поведения ошибочно отправлено на проверку 0 из 1 000 честных кандидатов, а модельных ботов — 100 % (500 из 500); условие: симуляция поведения: 1 000 модельных честных сессий и 500 модельных ботов; правило «два и больше сигнала или прямое доказательство», автоматической блокировки нет.

3.6 Регулярные задания и подписанный паспорт грейда

Работодатель собирает короткое задание из банка (или своё) и отправляет кандидатам категории; ответы проверяются автоматически и обновляют актуальность профиля. Результат теста можно предъявить вне платформы: паспорт грейда подписан электронной подписью Ed25519 и проверяется по открытому ключу на странице `/verify`.

4 Механика подбора и категоризация

4.1 Категории

Категория — связка «направление · грейд». Направления: Backend, Frontend, Data/ML, QA, информационная безопасность, DevOps/SRE, инженер данных, мобильная разработка, системный и бизнес-аналитик. Грейды: Стажёр, Junior, Middle, Senior, Lead, Principal. Кандидат попадает в категорию только по подтверждённому тестом грейду. Пока грейд не подтверждён, кандидат виден в подборке по направлению из опроса — последним ярусом и с пометкой «грейд не подтверждён»; заявленный грейд в скор не входит.

4.2 Разбор потребности

Работодатель пишет задачу своими словами («нужен бэкендер на платёжный сервис, Python, высокая нагрузка») или загружает файл вакансии. Платформа выделяет направление, грейд, обязательный и желательный стек и показывает цитату из текста для каждого требования. Условия, нарушающие статью 64 Трудового кодекса (пол, возраст, внешность и т. п.), отбрасываются с пояснением. Работодатель может поправить любое требование.

Разбор проверен на реальных вакансиях hh.ru: направление определено верно в 85,1 % (286 из 336). Условие: реальные вакансии hh.ru: второй отложенный набор: 336 свежих вакансий hh.ru (SHA-256 2bb77a1cd8d7..., заморожен 2026-10-21, прогон один); метки — поля hh.ru (роль и слова названия), разбор текста правилами без LLM; прогон 2026-10-21, код 249443367; N = 336; 21.10.2026.

4.3 Ранжирование и объяснение

Скор кандидата складывается из подтверждённых фактов (доли — из `backend/app/modules/matching/ranking.py`):

Составляющая	Доля скора
Результат теста по грейду и навыкам потребности	45 %
Обязательный стек потребности	35 %
Желательный стек	5 %
Подтверждённые достижения ФСП	10 %
Короткое задание работодателя (если было)	5 %

Правила, которые важны для честности выдачи:

- **Грейд — только подтверждённый.** Фильтр грейда и категории работает по результату теста, а не по резюме.
- **ФСП — бонус, а не штраф.** Достижение со статусом «подтверждено ФСП» даёт полный вес, «ждёт подтверждения» — половину, заявленное без документа — ноль. У кандидата без истории ФСП эта составляющая равна нулю без штрафа, и он остаётся в подборке на общих основаниях.
- **Объяснимость.** У каждого кандидата — короткое «почему он здесь»: какие навыки потребности подтверждены тестом и на каких заданиях, какие достижения ФСП учтены и с каким статусом, чего не хватает.
- **Внутри категории** выше тот, у кого сильнее результат теста и есть подтверждённые достижения ФСП (ТЗ 2.1).

На отложенных синтетических мирах доля подходящих кандидатов в первой пятёрке подборки (P@5) — 0,77 (95 % ДИ 0,74–0,80). Условие: модель: 210 потребностей × 900 модельных кандидатов на отложенных мирах (истина известна, не люди); миры заморожены до прогона, веса подбирались на других мирах; 21.10.2026.

4.4 Уточнение без потери подборки

Каждое изменение фильтров или требований создаёт новую версию подборки; прежние версии сохраняются, и между ними можно переключаться и сравнивать. Работодатель может включить «учитывать решения компании»: отметки «подходит / не подходит» мягко сдвигают веса скора, а изменение показывается явно. Подписка на потребность присылает уведомление, когда появляются новые подходящие кандидаты.

4.5 Выход на контакт

1. Работодатель открывает обезличенную карточку (подтверждённый грейд, навыки, факты ФСП, объяснение) и отправляет **приглашение**: описание предложения, зарплатная вилка «от–до» в рублях (обязательна, проверяется и на уровне базы данных), название компании, способ связи, по желанию — слоты собеседования. Можно сначала отправить **короткое задание** на 15 минут, чтобы получить свежий сигнал.
2. Кандидат видит условия до начала общения и принимает или отклоняет приглашение; повторное приглашение той же компанией тому же кандидату — не раньше чем через 30 дней.
3. Статусы «отправлено → просмотрено → принято / отклонено» видят обе стороны. Контакты кандидата раскрываются только после принятия приглашения или собственного отклика кандидата.
4. Взаимный подбор подсказывает работодателю до отправки, подойдёт ли предложение кандидату (формат, город, вилка против скрытого минимума кандидата — только «в ожиданиях / ниже», без раскрытия суммы).
5. После принятия — переписка, этапы найма и передача кандидата в ATS работодателя.

Классический сценарий тоже есть: работодатель публикует вакансию (с обязательной вилкой), кандидат откликается сам.

5 Процедура валидации и результаты

5.1 Принципы процедуры

Эталонной разметки в хакатоне нет, поэтому команда построила собственную процедуру. Её правила защищают от самообмана:

1. **Протокол до прогона.** Что измеряем, на каких данных и как выбираем настройки — записывается в репозиторий до отчётного прогона (`docs/validation/PRODUCT_SIM_PROTOCOL.md`, `docs/realdata/bench_holdout2_PROTOCOL.md`, `docs/realdata/djinni/PROTOCOL.md`).
2. **Разведённые данные.** Настройки подбираются только на обучающих зёрнах и мирах; отчётные зёрна, миры и наборы вакансий в настройке не участвуют. Отложенный набор замораживается, его хэш фиксируется коммитом до прогона.
3. **Один отчётный прогон.** Публикуется результат единственного прогона, каким бы он ни вышел; выбора лучшего из нескольких нет.
4. **Тот же код, что на сайте.** Симуляция гоняет продуктовый движок теста и подбора, а не упрощённую модель.
5. **Каждая цифра — с условием** (на ком, N, дата, команда пересчёта) и в одном файле `docs/NUMBERS.json`; на сайте, в документации и в презентации — одни и те же значения.
6. **Модель — не люди.** Где кандидаты модельные, это написано рядом с цифрой. Где метрика на модели недостижима честно, вместо неё публикуется более строгая (например, «верный итог проверки заявленного грейда» вместо «точного грейда»).



5.2 Главные метрики: стандартные условия и проверка на прочность

Каждая метрика публикуется дважды и рядом. Заголовок — **стандартные условия**, принятые в аналогичных решениях (тест: кандидаты отвечают по своему уровню, параметры заданий известны; подбор: точные резюме, грейд — по тесту). Сразу рядом — **проверка на прочность**: наши жёсткие условия — угадывание, завышенные заявки, кандидаты у самой границы грейда, неточные параметры заданий, неточные резюме. Какие условия идут заголовком, зафиксировано в протоколах до прогонов и от результатов не зависит; ни одна из двух строк не скрывается.

Метрика	Стандартные условия (как в аналогичных решениях)	Проверка на прочность (наши жёсткие условия)
Точность уверенного ответа	88,1 % (2 524 из 2 864) [1]	89,6 % (2 446 из 2 729) [2]
Точный грейд — 6 ступеней, все кандидаты	77,6 % (2 329 из 3 000) [1]	75,4 % (2 262 из 3 000) [2]
Грейд с точностью ±1 ступень	99,9 % (2 996 из 3 000) [1]	99,9 % (2 996 из 3 000) [2]
Повтор на других заданиях (взвешенная каппа)	0,85 [3]	0,91 [4]
Ложное подтверждение грейда (от всех)	2,1 % (63 из 3 000) [1]	2,7 % (82 из 3 000) [2]
Завышенная заявка прошла	8,8 % (63 из 714) [1]	7,1 % (82 из 1 160) [2]
Подбор: точность первой пятёрки (P@5)	77,1 % [5]	75,9 % [6]
Подбор: nDCG@10	0,66 [5]	0,57 [6]
Подбор: MRR	0,86 [5]	0,88 [6]
Подбор: хотя бы один подходящий в первой пятёрке	98,1 % [5]	98,5 % [6]
Подбор: переоценённых в первой десятке	2,8 % [5]	4,0 % [6]

Реальные данные	Значение
Вакансии hh.ru: направление определено верно	85,1 % (286 из 336) [7]
Открытый набор вакансий — nDCG@10	0,58 [8]
Официальных протоколов ФСП в индексе	18 [9]
Результатов спортсменов в индексе	2 908 [9]

1. Стандартные условия (как в аналогичных решениях): a = 1,5 в движке, истинные a × U(0,825; 1,175) и b + N(0; 0,3), угадывание 0,25 в вопросах с выбором и 0,02 в остальных, θ ~ N(0,5; 1,0), заявки: 65 % точные, 25 % завышены, 10 % занижены; модельные кандидаты, не люди; N = 3 000, зёрна 5501…5510; время — в среднем 43 мин (p90 69); банк ab16d27 (913ad19f6a259aff), 2026-10-21. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase st --n 300 --seeds 5501 --out /w/a4fin2/out/b/S_T__5501.json --population st --world st --bank-a mcq=1.5,code=1.5 --retest --fixed --expect-bank-hash 913ad19f6a259aff`.
2. Жёсткие условия: угадывание в вопросах с выбором, завышение и занижение заявок, кандидаты у порогов, консервативные параметры банка (a = 1,0 / 1,3); модельные кандидаты, не люди; N = 3 000, зёрна 4401…4410; время — в среднем 45 мин (p90 81); банк ab16d27 (913ad19f6a259aff), 2026-10-21. Пересчёт: `cd backend && python -m`

`app.modules.validation.product_sim --phase final --n 300 --seeds 4401 --out /w/a4fin2/out/main/F_calibrated__4401.json --retest --fixed --expect-bank-hash 913ad19f6a259aff`.

3. Стандартные условия (как в аналогичных решениях): a = 1,5 в движке, истинные a × U(0,825; 1,175) и b + N(0; 0,3), угадывание 0,25 в вопросах с выбором и 0,02 в остальных, θ ~ N(0,5; 1,0), заявки: 65 % точные, 25 % завышены, 10 % занижены; модельные кандидаты, не люди; N = 3 000, зёрна 5501…5510; время — в среднем 43 мин (p90 69); банк ab16d27 (913ad19f6a259aff), 2026-10-21; пар повтора 3 000. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase st --n 300 --seeds 5501 --out /w/a4fin2/out/b/S_T__5501.json --population st --world st --bank-a mcq=1.5,code=1.5 --retest --fixed --expect-bank-hash 913ad19f6a259aff`.
4. Жёсткие условия: угадывание в вопросах с выбором, завышение и занижение заявок, кандидаты у порогов, консервативные параметры банка (a = 1,0 / 1,3); модельные кандидаты, не люди; N = 3 000, зёрна 4401…4410; время — в среднем 45 мин (p90 81); банк ab16d27 (913ad19f6a259aff), 2026-10-21; пар повтора 3 000. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase final --n 300 --seeds 4401 --out /w/a4fin2/out/main/F_calibrated__4401.json --retest --fixed --expect-bank-hash 913ad19f6a259aff`.
5. Стандартные условия, принятые в аналогичных решениях: 3 отложенных мира × 70 потребностей × 900 кандидатов, 9 направлений; тест у всех (погрешность 0,363 — средняя SE нашего теста), в 35 % резюме по 4 ложных навыка; метка 0–3 (точный грейд и ≥ 75 % обязательных навыков — 3); модели, не люди.
6. Жёсткие условия («проверка на прочность»): 2 отложенных мира × 200 потребностей × 2 000 кандидатов; 20 % анкет завышают грейд и стек, погрешность теста 0,48, люди у порогов; модели, не люди.
7. Второй отложенный набор: 336 свежих вакансий hh.ru (SHA-256 2bb77a1cd8d7…, заморожен 2026-10-21, прогон один); метки — поля hh.ru (роль и слова названия), разбор текста правилами без LLM; прогон 2026-10-21, код 249443367.
8. Внешний открытый набор, метки-посредники (категория и стаж из анкеты), не люди; режим «без теста»; N = 50 вакансий × 200 анкет; один прогон после FREEZE.
9. Официальные протоколы ФСП России 2025–2026 с сайта федерации; судьи, тренеры и справки исключены; 21.10.2026. Пересчёт: `python3 tools/fsp_protocols/export_public.py /opt/spectrek_fsp/data_local/fsp_protocols/index_v7.json docs/realdata/fsp_protocols backend/app/modules/fsp`.

5.3 Тест на грейд

Модельные кандидаты с известным истинным уровнем проходят тот же движок, что на сайте, на реальном банке заданий. Проверяются три «мира»: параметры заданий известны точно; трудность заданий известна неточно (банк ещё не откалиброван на людях, движок об этом не знает); проверка развёрнутых ответов ошибается. Повторяемость — второе прохождение тех же кандидатов на другом наборе заданий.

Что измерено	Значение	Условие
Верный итог проверки заявленного грейда	84,8 % (2 544 из 3 000); 95 % ДИ 83,5–86,0 %	[1]
Кандидатов получают уверенный ответ уже по основному тесту	89,2 % (2 676 из 3 000); 95 % ДИ 88,0–90,3 %	[1]
Точность уверенного ответа	89,8 % (2 403 из 2 676); 95 % ДИ 88,6–90,9 %	[1]

Что измерено	Значение	Условие
Кандидатов направляется на короткий дополнительный этап	10,8 % (324 из 3 000); 95 % ДИ 9,7–12,0 %	[1]
Грейд определён точно	77,6 % (2 329 из 3 000); 95 % ДИ 76,1–79,1 %	[1]
Ложных подтверждений от всех кандидатов	2,1 % (63 из 3 000); 95 % ДИ 1,6–2,7 %	[1]
Ошибок больше чем на одну ступень	0,1 % (4 из 3 000); 95 % ДИ 0,1–0,3 %	[1]
Совпадение итога при повторном прохождении	72,9 % (2 187 из 3 000); 95 % ДИ 71,3–74,5 %	[2]
Совпадение грейда при повторном прохождении	68,5 % (2 056 из 3 000); 95 % ДИ 66,8–70,2 %	[2]
Верный итог при неточно известной трудности заданий	84,4 % (1 688 из 2 000); 95 % ДИ 82,7–85,9 %	[3]
Верный итог при ошибках проверки развёрнутых ответов	86,1 % (1 292 из 1 500); 95 % ДИ 84,3–87,8 %	[4]
Среднее время теста	43,2 мин	[1]
Время теста у 90 % кандидатов не больше	69,4 мин	[1]

1. Стандартные условия (как в аналогичных решениях), зёрна 5501–5510; симуляция продукта: модельные кандидаты, не люди; N = 3 000; тот же движок теста, что на сайте; банк ab16d27 (1 646 семейств заданий); 21.10.2026. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase st --n 300 --seeds 5501 --out /w/a4fin2/out/b/S_T__5501.json --population st --world st --bank-a mcq=1.5,code=1.5 --retest --fixed --expect-bank-hash 913ad19f6a259aff`.
2. Стандартные условия (как в аналогичных решениях), зёрна 5501–5510; симуляция продукта: модельные кандидаты, не люди; N = 3 000; тот же движок теста, что на сайте; банк ab16d27 (1 646 семейств заданий); 21.10.2026; повтор на другом наборе заданий. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase st --n 300 --seeds 5501 --out /w/a4fin2/out/b/S_T__5501.json --population st --world st --bank-a mcq=1.5,code=1.5 --retest --fixed --expect-bank-hash 913ad19f6a259aff`.
3. Симуляция продукта, модельные кандидаты; N = 2 000; трудность и различающая сила заданий в «мире» сдвинуты случайно, движок об этом не знает; 21.10.2026. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase final --n 200 --seeds 4401 --out /w/a4fin2/out/main/F_misspec__4401.json --world misspec --expect-bank-hash 913ad19f6a259aff`.
4. Симуляция продукта, модельные кандидаты; N = 1 500; проверка развёрнутых ответов ошибается случайно; 21.10.2026. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase final --n 150 --seeds 4401 --out /w/a4fin2/out/main/F_rater__4401.json --rater-noise 0.1 --expect-bank-hash 913ad19f6a259aff`.

Почему главная метрика — верный итог проверки, а не «точный грейд». Работодателю нужен ответ «подтверждён ли заявленный уровень». Угадать точную ступень у кандидата, чей уровень лежит у самой границы, за один тест математически нельзя ни одним методом; поэтому у границы платформа честно назначает второй этап, а не угадывает.

5.4 Банк заданий

Что измерено	Значение	Условие
Семейств заданий в банке	1 646	[1]
Семейств с кодом и развёрнутым ответом	416	[1]
Направлений	10	[1]
Уникальных вариантов на проверке	32 896	[1]
Автоматических проверок ответов	38 131	[1]
Проверок с ошибкой	0	[1]

1. Автоматическая проверка банка: 20 вариантов-зёрен на каждое семейство, коммит банка ab16d27, 21.10.2026. Пересчёт: `cd backend && python -m assessment_bank validate --seeds 20`.

5.5 Подбор

Синтетические миры: потребности и модельные кандидаты с заранее известной релевантностью; веса подбирались на других мирах; сравнение — с поиском по тексту резюме и с подбором без отдельных составляющих.

Что измерено	Значение	Условие
Доля подходящих в первой пятёрке подборки (P@5)	0,77; 95 % ДИ 0,74–0,80	[1]
MRR — насколько высоко первый подходящий кандидат	0,86	[1]

1. Модель: 210 потребностей × 900 модельных кандидатов на отложенных мирах (истина известна, не люди); миры заморожены до прогона, веса подбирались на других мирах; 21.10.2026. Пересчёт: `python3 tools/realdata/bench_holdout4.py run (один раз после freeze; повтор запрещён) → python3 tools/realdata/numbers_matching.py`.

Реальные вакансии hh.ru: насколько верно платформа понимает живой текст вакансии — направление, грейд, навыки. Набор отложенный: собран после настройки словаря и разбора и заморожен до прогона.

Что измерено	Значение	Условие
Верно определено направление вакансии	85,1 % (286 из 336)	[1]

1. Реальные вакансии hh.ru: второй отложенный набор: 336 свежих вакансий hh.ru (SHA-256 2bb77a1cd8d7..., заморожен 2026-10-21, прогон один); метки — поля hh.ru (роль и слова названия), разбор текста правилами без LLM; прогон 2026-10-21, код 249443367; N = 336; 21.10.2026. Пересчёт: `cd backend && python ../tools/realdata/hh_holdout.py --tag 2 run (один раз; новый прогон — новый отложенный набор: collect → freeze → run)`.

Что можно пересчитать из репозитория, а что нет. Тексты вакансий hh.ru по правилам площадки в репозиторий не кладутся: они хранятся на сервере команды вне git. В репозитории — номера вакансий, ссылки, метки и контрольные суммы текстов (sha256): `docs/realdata/hh_match_corpus.json`. Пересчёт `python tools/realdata/hh_plus.py match` работает при наличии этого кэша текстов; без него тексты можно скачать заново по номерам через API hh.ru и сверить контрольные суммы, но вакансии, снятые с публикации, восстановить нельзя. Спрос по направлениям и зарплатные

вилки рынка — снимок API hh.ru на 10.10.2026 (поле `as_of` в `backend/app/modules/hh/data/demand.json` и `market_snapshot.json`): на другую дату числа будут другими. Вилки — объявления работодателей, а не фактические зарплаты.

5.6 Защита проверки ответов и честность прохождения

Что измерено	Значение	Условие
Атак перехвачено фильтром до оценки моделью	77,8 % (137 из 176)	[1]
Ложных срабатываний на честных ответах	1,1 % (2 из 176)	[1]
Атак прошли всю цепочку защиты	0 из 176	[2]
Честных кандидатов ошибочно отправлено на проверку	0 из 1 000	[3]
Модельных ботов отправлено на дополнительную проверку	100 % (500 из 500)	[3]

1. Отложенный набор атак на проверку ответов, составлен после фиксации кода фильтра (коммит 6b7c710c), один прогон без донастройки; N = 176 атак и 176 честных ответов; 20.10.2026. Пересчёт: `python3 tools/grader_check/run_holdout_eval.py --offline`.
2. Отложенный набор атак на проверку ответов, составлен после фиксации кода фильтра (коммит 6b7c710c), один прогон без донастройки; N = 176 атак и 176 честных ответов; 20.10.2026; полная цепочка проверки. Пересчёт: `python3 tools/grader_check/run_holdout_eval.py --llm --max-cost-usd 3.0`.
3. Симуляция поведения: 1 000 модельных честных сессий и 500 модельных ботов; правило «два и больше сигнала или прямое доказательство», автоматической блокировки нет. Пересчёт: `python3 tools/grader_check/run_holdout_eval.py --offline`.

5.7 Реальные документы ФСП

Что измерено	Значение	Условие
Официальных протоколов в индексе	18	[1]
Соревнований	27	[1]
Результатов	2 908	[1]
Спортсменов	2 232	[1]
Субъектов РФ	69	[1]
Спортсменов с выполненным нормативом ЕВСК	130	[1]

1. Официальные протоколы ФСП России 2025–2026 с сайта федерации; судьи, тренеры и справки исключены; 21.10.2026. Пересчёт: `python3 tools/fsp_protocols/export_public.py /opt/spectrek_fsp/data_local/fsp_protocols/index_v7.json docs/realdata/fsp_protocols backend/app/modules/fsp`.

5.8 Ограничения

- Калибровки трудности заданий на большой выборке живых людей пока нет: симуляция «неточно известной трудности» показывает, насколько результат устойчив к этому. Обезличенная статистика прохождения собирается на сайте; калибровка на людях — следующий шаг, план пилота ниже.

- Разметка — моделями, а не экспертами-людьми: релевантность в синтетических мирах задана генератором миров; на реальных вакансиях проверяется разбор потребности, а не итоговое решение о найме.
- Метки hh.ru сами содержат шум; спорные случаи перепроверены независимыми арбитрами-моделями (разметка моделями, а не людьми), и это указано рядом с цифрой.

5.9 План пилота с ФСП

Пилот ещё не проведён: ниже — план, а не результат. Его цель — заменить модельные допущения измерениями на живых кандидатах и тем же порядком: протокол до начала, отложенные данные, один отчётный расчёт.

Что	План
Участники	Спортсмены ФСП и студенты профильных вузов, 300–500 человек, 2–3 направления; добровольно, с согласием на обработку данных
Срок	6–8 недель после финала
Калибровка заданий	Трудность и различающая сила заданий пересчитываются по ответам участников; задания с плохим согласием с моделью снимаются с выдачи
Независимая проверка вердикта	Для 100 участников, в первую очередь у границы грейдов, — слепое техническое собеседование двумя экспертами, которые не видят результат теста; сравнение вердикта теста с их решением и согласие экспертов между собой
Подбор	3–5 компаний-партнёров ФСП с реальными потребностями: доля приглашённых кандидатов, дошедших до собеседования, и доля отказов из-за несоответствия уровня
Порядок	Протокол, метрики и правило расчёта фиксируются коммитом в репозитории до начала; половина ответов — для калибровки, половина — отложена для отчёта; один отчётный расчёт
Данные	Только обезличенная статистика ответов; персональные данные — по 152-ФЗ, в отчёт не попадают

6 Интеграция с реестром ФСП

ТЗ не даёт API и структуры данных ФСП и просит команду предложить, какие сведения получать и как показывать. «Драфт» работает с тремя источниками — от самого надёжного к временному.

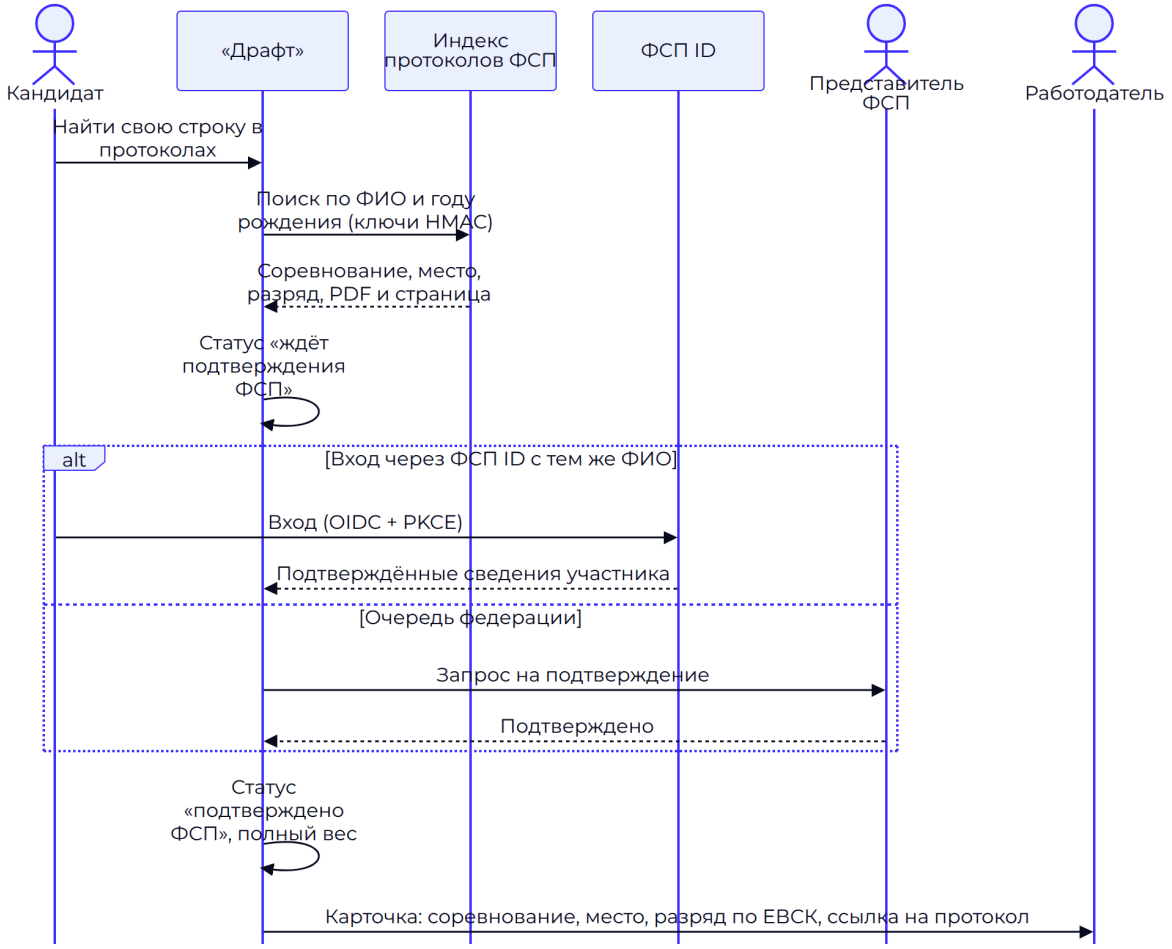
Источник	Что даёт	Статус в продукте
ФСП ID (OIDC, Keycloak)	вход и связь профиля с ID участника; разряд, дисциплины, достижения в составе учётной записи	работает на демо-стенде Keycloak; переключение на настоящий ФСП ID — смена адреса, клиента и секрета без изменения кода
Официальные протоколы ФСП России	места, разряды, номер соревнования в Едином календарном плане, ссылка на PDF и страницу	индекс протоколов с сайта федерации; кандидат находит свою строку, работодатель видит ссылку на документ
API реестра ФСП (предложение контракта)	снимок достижений участника по его ФСП ID	контракт `docs/fsp_registry_api.yaml` и рабочий адаптер; режим `external` включается переменными окружения

Индекс содержит 18 официальных протоколов: 2 908 результатов, 2 232 спортсменов из 69 субъектов РФ (официальные протоколы ФСП России 2025–2026 с сайта федерации; судьи, тренеры и справки исключены; 21.10.2026).

6.1 Статусы достижения

Статус	Как получается	Вес в составляющей ФСП
Нет истории ФСП	кандидат не указал достижений	0, без штрафа; фраза в карточке: «Истории ФСП нет — без штрафа»
Заявлено	кандидат указал достижение без документа	0
Ждёт подтверждения ФСП	кандидат нашёл свою строку в официальном протоколе	половина
Подтверждено ФСП	совпадение через ФСП ID или решение представителя федерации	полный

Судейские и тренерские записи и строки справок не считаются достижениями. Разряд по Единой всероссийской спортивной классификации (ЕВСК) выводится из места, числа участвующих субъектов РФ и состава судейской коллегии и показывается с основанием.



6.2 Персональные данные спортсменов

Обработка сверена с Положением ФСП об обработке персональных данных. Из протокола берутся только ФИО, год рождения, субъект РФ, команда, место и разряд. Индекс хранится вне репозитория, доступен только процессу API; ключи поиска — HMAC с серверным секретом. Чужие ФИО никому не показываются: работодатель видит только результат сверки по кандидату, который сам дал согласие. Открытого каталога спортсменов нет; публично — только обезличенные агрегаты (по регионам и дисциплинам) в кабинете федерации. Для несовершеннолетних отправка и принятие оффера требуют отметки согласия законного представителя и особого формата работы.

6.3 Как перейти на боевую интеграцию

1. Зарегистрировать «Драфт» клиентом ФСП ID (Authorization Code + PKCE), согласовать состав атрибутов участника; различия в именах задаются настройкой `FSP_OIDC_CLAIMS_MAP` без изменения кода.
2. Когда появится API реестра — включить `FSP_REGISTRY_MODE=external` с адресом и сервисным токеном; контракт `docs/fsp_registry_api.yaml` — предложение команды для обсуждения с федерацией.
3. Индекс протоколов остаётся запасным источником для участников без ФСП ID.

Подробно: `docs/FSP_INTEGRATION.md`, `docs/realdata/fsp_protocols/README.md`.

7 Описание API

Все методы описаны спецификацией OpenAPI 0.2.0: 438 операций в 42 группах. Спецификация собирается из кода (`make openapi`) и проверяется тестом на актуальность.

- Интерактивное описание: https://fsp-draft.ru/api/docs (Swagger UI), https://fsp-draft.ru/api/redoc; файлы интерфейса документации раздаются с нашего сервера, внешних CDN нет.
- Файл: `docs/api/openapi.json`, на сайте — `/api/openapi.json`.
- Аутентификация: `Authorization: Bearer <JWT>`; токен выдаёт `POST /api/auth/login` после подтверждения почты; служебные роли — со вторым фактором. Права проверяются по роли на каждом методе; матрица прав проверяется автотестом.
- Ошибки — единый формат `{"detail": {"code": "<машинный код>", "message": "<текст по-русски>"}}`; коды ошибок каждого метода перечислены в спецификации, у ответов есть примеры.

Группа (тег OpenAPI)	Что делает	Операций
`interview`	Устная защита развёрнутых ответов теста и ИИ-интервью этапом найма (флаг INTERVIEW_ENABLED).	32
`validation`	Публичные метрики качества: валидация теста и подбора, критерии ТЗ, проверка своих данных	30
`ats`	Передача кандидатов в ATS: Хантфлоу API v2, Talantix GraphQL, «Поток» REST v3, вебхук с HMAC, экспорт JSON, PDF-профиль по подписанной ссылке	23
`employer`	Работодатель: потребности, подборки с версиями, поиск, карточки кандидатов	23
`ats-stub`	Демо-заглушки ATS (Хантфлоу, Talantix, «Поток») по настоящему HTTP	19
`candidate`	Кандидат: опрос, согласия 152-ФЗ, приватность, ФСП ID, резюме, PDF-профиль	17
`auth`	Регистрация, подтверждение почты, вход (JWT), второй фактор (TOTP)	16
`cups`	«Кубок работодателя»: соревнование в категории на базе зонда, таблица результатов под псевдонимами, приглашения лучшим, запись в паспорт ФСП	16
`hiring`	Найм после принятия приглашения: слоты собеседования, оффер со встречным, исход найма; этапы уходят в ATS работодателя и возвращаются обратно	16

Группа (тег OpenAPI)	Что делает	Операций
`fsp-cabinet`	Кабинет ФСП: спрос работодателей, эффект соревнований ФСП, реальные зарплатные вилки, воронки — только агрегаты (минимальная группа 5)	16
`vacancies`	Вакансии и отклики кандидатов	15
`passport`	Подписанный паспорт грейда (Ed25519): публичная проверка, ключи, отзыв	14
`assignments`	Задания работодателя с автопроверкой: сборка 3–5 заданий банка по стеку потребности или вакансии, зонд кандидату, задание к вакансии, лента «Задания для вас»; результат — факт T1 по навыку	14
`mutual`	Взаимный подбор (H9): насколько предложение подходит кандидату; обратная связь по подборке и новые версии (H2); ссылка для нанимающего руководителя — только чтение, без ПДн, 7 дней (H3)	14
`assessment`	Тест на грейд и экран «Категория и грейд» (правило 90 дней, демо-режим жюри)	13
`jury-live`	Живая демонстрация для жюри: QR → мини-тест на телефоне → экспресс-категория → подборка → автоприглашение с push	12
`admin`	Мини-админка (/admin): пользователи, компании, заявки на тарифы и партнёрство, обращения, флаги и воронка продукта	11
`subscriptions`	Подписка на потребность («новые подходящие»), уведомления кабинета, напоминания	10
`candidate-contact`	Кандидат: зонды и приглашения, принятие и отказ	9
`contact`	Переписка по приглашению и отклику, встречное предложение кандидата, отзыв доступа к контактам	9
`proctor`	Камера во время теста по согласию: анализ только в браузере, на сервер — события без кадров; без камеры тест доступен полностью	9

Группа (тег OpenAPI)	Что делает	Операций
`study`	Обезличенная статистика прохождения у жюри и посетителей стенда: уведомление и отказ, путь, необязательная анкета SUS и оценки топ-5, сводка для /validation (при N ≥ 3)	9
`trio`	Трёхсторонняя связь по приглашению: работодатель ↔ кандидат ↔ представитель ФСП (согласие кандидата, маскировка контактов до принятия)	9
`expert-panel`	Экспертная разметка: вход по ссылке, пары, ответы, Ангофф	7
`jury`	Жюри: вход в один клик, личная песочница с демо-данными, сброс и удаление песочницы	7
`warroom`	Боевой ИИ-Полигон: детерминированный симулятор архитектурных инцидентов и проверка античита	7
`employer-contact`	Работодатель: зонды и приглашения-офферы (вилка обязательна, лимит 30 дней)	6
`tariffs`	Тарифы работодателя: «Старт» (бесплатно), «Найм» (цена по запросу, демо-счёт без суммы), «Партнёр ФСП» (выдаёт ФСП); лимит приглашений в месяц.	6
`my-data`	«Мои данные»: что хранится о пользователе, выгрузка JSON, удаление, отзыв раскрытых контактов	6
`sus`	Опрос юзабилити SUS и агрегаты для /validation	5
`employer-trust`	Проверка работодателя в ЕГРЮЛ (ФНС) и индекс доверия	5
`fsp-id`	Вход и связка через ФСП ID (демо, Keycloak, OIDC + PKCE)	5
`push`	Web Push (VAPID) для PWA	5
`telegram`	Уведомления в Telegram: привязка по одноразовому коду и быстрые ответы на приглашения	5
`hh`	hh.ru: импорт своей вакансии работодателя (только чтение GET /vacancies/{id}) и «Рынок по hh.ru» — медиана и p25–p75 вилок живых вакансий.	3
`expert`	Проверка развёрнутых ответов теста по рубрике: очередь, оценка, согласованность экспертов	3

Группа (тег OpenAPI)	Что делает	Операций
`profiles`	Профиль текущего пользователя: кандидат или работодатель	3
`support`	Служба поддержки и обратная связь (/support): приём обращений и история своих заявок	3
`catalog`	Справочник направлений, грейдов, категорий и стек-тегов	2
`system`	Служебные методы	2
`fsp-registry`	Предложение контракта реестра ФСП и его синтетический мок (не официальный API ФСП)	1
`legal`	Тексты согласий и реквизиты оператора персональных данных	1

Главные методы сквозного сценария:

Шаг	Метод
Регистрация и подтверждение почты	`POST /api/auth/register`, `POST /api/auth/verify-email`
Вход	`POST /api/auth/login`
Опрос: направление, стек, грейд	`PUT /api/me/onboarding`
Профиль из PDF-резюме	`POST /api/me/resume`
Тест на грейд	`POST /api/assessment/sessions`, `POST /api/assessment/sessions/{session_id}/answers`, `POST /api/assessment/sessions/{session_id}/finish`
Категория, грейд, история	`GET /api/assessment/status`
Потребность и подборка	`POST /api/employer/needs`, `POST /api/employer/needs/file`, `POST /api/employer/needs/{need_id}/selections`
Поиск по базе	`GET /api/employer/candidates`
Приглашение	`POST /api/employer/invitations`
Ответ кандидата	`POST /api/candidate/invitations/{inv_id}/accept`, `POST /api/candidate/invitations/{inv_id}/decline`
Вакансии и отклик	`GET /api/vacancies`, `POST /api/vacancies/{vid}/apply`
Сверка с протоколами ФСП	`POST /api/me/fsp/protocols/search`, `POST /api/me/fsp/protocols/confirm`

Тела запросов и ответы — в спецификации. Каждый метод из этой таблицы при сборке документа сверяется со спецификацией: несуществующий путь — ошибка сборки.

8 Библиотеки и компоненты

Версии ниже собраны автоматически из файлов зависимостей репозитория при сборке документа. Все библиотеки — с открытыми лицензиями; полный список с лицензиями — `THIRD_PARTY.md`.

Backend (Python, `backend/requirements.txt`, версии зафиксированы):

Библиотека	Версия
fastapi	0.142.2
uvicorn[standard]	0.54.0
SQLAlchemy	2.1.3
alembic	1.20.0
pydantic	2.13.5
pydantic-settings	2.15.0
email-validator	2.3.0
argon2-cffi	25.1.0
PyJWT	2.15.1
python-multipart	0.0.32
psycopg[binary]	3.3.6
PyYAML	6.0.3
boto3	1.43.108
httpx	0.28.1
pypdf	6.19.0
reportlab	5.0.1
natasha	1.6.0
numpy	2.1.3
scipy	1.14.1
cryptography	50.0.2
pywebpush	2.5.0

Frontend (`frontend/package.json`, установленные версии — из `package-lock.json`):

Пакет	Назначение	Версия
@anam-ai/js-sdk	в сборке сайта	4.28.0
@fontsource/montserrat	в сборке сайта	5.3.0
react	в сборке сайта	18.3.1
react-dom	в сборке сайта	18.3.1
react-router-dom	в сборке сайта	6.30.6
@types/node	сборка и проверка типов	20.19.43
@types/react	сборка и проверка типов	18.3.31
@types/react-dom	сборка и проверка типов	18.3.7

Пакет	Назначение	Версия
@vitejs/plugin-react	сборка и проверка типов	4.7.0
autoprefixer	сборка и проверка типов	10.6.1
postcss	сборка и проверка типов	8.5.29
tailwindcss	сборка и проверка типов	3.4.19
typescript	сборка и проверка типов	5.9.3
vite	сборка и проверка типов	5.4.21

Образы контейнеров:

Где	Образ
`backend/Dockerfile`	`python:3.12-slim`
`frontend/Dockerfile`	`node:20-alpine`
`frontend/Dockerfile`	`nginx:1.27-alpine`
`keycloak/Dockerfile`	`quay.io/keycloak/keycloak:26.4.0`
`keycloak/Dockerfile`	`quay.io/keycloak/keycloak:26.4.0`
`docker-compose.yml`	`${LLM_LOCAL_IMAGE:-ghcr.io/ggml-org/llama.cpp:server-b11515}`
`docker-compose.yml`	`postgres:16-alpine`

9 Сборка, развёртывание и локальный запуск

9.1 Требования

- Linux, macOS или Windows с Docker Engine и Docker Compose v2.24+, `make`, `git`.
- Для запуска без Docker — Python 3.12+, Node.js 20+.
- Внешние ключи не обязательны: без почтового сервиса коды подтверждения пишутся в лог, без языковой модели работают правила и очередь экспертов.

9.2 Запуск в Docker (рекомендуется)

```
git clone https://github.com/neuropeopleteam-afk/draft.git
cd draft
make up        # создаёт .env со случайными секретами, собирает образы, применяет миграции,
        создаёт демо-данные
               # NO_SEED=1 make up — без демо-данных
make ps        # состояние контейнеров
make logs      # логи API; в режиме разработки здесь видны коды подтверждения почты
```

Сайт — http://localhost:8080, описание API — http://localhost:8080/api/docs. Остановить — `make down` (данные БД сохраняются в томе `db_data`).

Дополнительные профили:

```
COMPOSE_PROFILES=fsp-id make up   # + демо-стенд «ФСП ID» (Keycloak)
make llm-local                    # + открытая языковая модель на CPU, без внешних ключей
```

9.3 Запуск без Docker

```
make dev-backend    # API на SQLite с автоперезагрузкой, http://127.0.0.1:8000
make dev-frontend   # интерфейс Vite с прокси /api → :8000, http://127.0.0.1:5173
```

9.4 Проверка

```
make test           # тесты backend в контейнере (SQLite + миграции)
make test-pg        # те же тесты на PostgreSQL
make test-bank      # валидатор банка заданий
make openapi        # пересборка docs/api/openapi.json из кода
python3 tools/publish/gen_numbers.py   # пересборка docs/NUMBERS.json из отчётов прогонов
python3 docs/submission/build.py       # пересборка README, документации и презентации
```

9.5 Развёртывание на сервере

1. Сервер с Docker (для стенда достаточно небольшого виртуального сервера; лимиты памяти контейнеров заданы в `docker-compose.yml`).
2. `git clone`, `make env`, заполнить в `.env` публичный адрес (`PUBLIC_BASE_URL`), почту отправителя и, при необходимости, ключи внешних сервисов — их список и назначение описаны в `.env.example`. Секреты хранятся вне репозитория с правами 600.
3. `make up` — контейнеры слушают только `127.0.0.1`; снаружи их публикует веб-сервер хоста с TLS. Пример конфигурации — `infra/`.

4. Миграции применяются автоматически при старте контейнера `api`; демо-данные — идемпотентно.
5. Обновление: `git pull && make up` (пересборка образов и миграции).

10 Нефункциональные требования и безопасность

Требование ТЗ 3.5	Как выполнено
Открытый код	исходный код открыт, не скомпилирован и не обфусцирован; лицензия MIT; зависимости — открытые
Архитектура	frontend (React) · backend (FastAPI, модульный монолит со слоями) · PostgreSQL; модули связаны через сервисы
Контейнеризация	Dockerfile для `api` и `web`, `docker-compose.yml`, запуск одной командой `make up`
API	OpenAPI со всеми методами, примерами и кодами ошибок (раздел 7)
Аутентификация	собственные учётные записи + вход через ФСП ID по OIDC (Keycloak)
Платформенность	веб для современных браузеров, адаптивная вёрстка 360 / 768 / 1280 px, установка как приложение, Android-приложение
Качество кода	комментарии к сложным решениям, тесты backend и сквозные тесты интерфейса, проверка в CI

Безопасность и 152-ФЗ.

- Согласия на обработку, на публикацию профиля и на камеру во время теста — отдельные, с датой и версией текста; без согласия на публикацию кандидат не виден работодателям; отозвать согласие можно в любой момент.
- Работодатель видит обезличенную карточку; контакты — только после принятия приглашения или отклика.
- Перед отправкой текста внешней модели персональные данные вырезаются.
- Пароли — argon2id; коды подтверждения хранятся как HMAC, с ограничением попыток и сроком; JWT отзывается при смене пароля; второй фактор для служебных ролей; ключи ATS хранятся зашифрованными и имеют срок действия.
- Права проверяются по роли на каждом методе; кандидат видит только свои приглашения и отклики.
- «Мои данные»: что хранится о пользователе, выгрузка в JSON, удаление аккаунта с закрытием выданных контактов.
- Протоколы ФСП — вне репозитория, ключи поиска HMAC, без открытого каталога спортсменов (раздел 6).

11 Монетизация

Кандидатам — бесплатно. Платят работодатели и партнёры федерации — за подтверждённый найм:

- **Тарифы работодателей:** «Старт» — бесплатно с ограниченным числом приглашений; «Найм» — для регулярного подбора с интеграцией ATS, заданиями с автопроверкой и аналитикой зарплатных вилок; «Партнёр ФСП» — в составе спонсорского пакета федерации.
- **Спонсорские пакеты ФСП:** партнёр соревнования, партнёр направления, генеральный партнёр — отборочные задачи компании в программе соревнований, кубки работодателя, прямые приглашения призёрам, аналитика кадрового резерва.

Цены — по запросу, оформление через заявку на странице тарифов (`/pricing`). Модель выгодна федерации: соревнования становятся источником найма, а спортсмены получают карьерное преимущество от своих результатов.

Приложение А. Соответствие ТЗ

Каждый пункт ТЗ — как сделан и где его проверить. Сайт — путь на https://fsp-draft.ru (в демо-роль входят в один клик из `/demo`); API — метод в https://fsp-draft.ru/api/docs; тест — файл в `backend/tests/` (запуск `make test`) или сквозной тест интерфейса в `e2e/tests/`; «док.» — раздел этого документа.

Пункт ТЗ	Как сделано	Где проверить
2 Обратная механика: работодатель сам выходит на кандидата	Работодатель описывает потребность, получает подборку и отправляет приглашение с вилкой; кандидат не откликается вслепую	сайт `/employer/needs/<id>` → «Пригласить»; тест `e2e/tests/02-employer-invite.spec.ts`; док. 4
2 Категория — из опроса, выбора грейда и теста, а не из резюме	Категория «направление · грейд» появляется только после теста; без опроса кандидат в выдаче не показывается	сайт `/candidate/assessment`, `/candidate/category`; тесты `test_status.py` (`test_unconfirmed_without_survey_not_listed`), `e2e/tests/01-candidate-test.spec.ts`; док. 3
2 Задания не расходятся между участниками рынка; сложность сопоставима	Параметрические семейства заданий: у каждого кандидата свои варианты одной калиброванной сложности; ключи ответов не уходят в браузер	тест `e2e/tests/21-no-answer-keys.spec.ts`; валидатор `make test-bank`; док. 3.3
2 Грейд не понижается принудительно; смена грейда ограничена по времени	Провал не снимает подтверждённый грейд; тест ниже — сразу, тот же или выше — через 30 дней, смена грейда — не чаще раза в 90 дней	сайт `/candidate/category`; тесты `test_day2_rules.py` (`test_failed_test_does_not_downgrade`), `test_jury_test.py` (`test_retake_cooldown_api`); код `backend/app/core/config.py`
2 Регулярные короткие задания от работодателя	Работодатель создаёт задание, кандидат решает его или предлагает подход; результат обновляет актуальность профиля	сайт `/employer/assignments`, `/candidate/assignments`; тест `test_assignments_targeted.py`
2 Связь профиля с ФСП ID; совместимость с Keycloak	Вход и привязка по стандартному протоколу единого входа; демо-стенд «ФСП ID» на Keycloak	сайт `/candidate/fsp`; API `/api/fsp-id/start`; тест `test_integrations.py` (`test_fsp_id_new_user_consent_and_passport_t2`); папка `keycloak/`; док. 6
2 Корректная работа без истории ФСП	Составляющая ФСП равна нулю, без штрафа; кандидат в выдаче на общих основаниях	сайт `/demo` → профиль без истории ФСП; тест `test_integrations.py` (`test_fsp_id_link_existing_and_no_history`); док. 4, 6
2 Собственная процедура валидации и её результаты	Протокол фиксируется до прогона, итог — один прогон на отложенных данных; каждая цифра — с условием и командой пересчёта	сайт `/validation`; файлы `docs/validation/`, `docs/NUMBERS.json`; док. 5

Пункт ТЗ	Как сделано	Где проверить
2 (плюс) Защита от фиктивных вакансий и недобросовестных работодателей	Проверка компании по реестру, контроль вилки по рынку, жалобы кандидатов	сайт `/employer/company`; API `/api/employer/company/registry-check`; `docs/ANTIFRAUD.md`; тест `e2e/tests/05-invitation-complaint.spec.ts`
2 (плюс) Интеграция с ATS работодателя	Передача кандидата и обратная синхронизация этапов найма; сбой ATS не блокирует найм	сайт `/employer/ats`; `docs/ATS.md`; тесты `test_ats_stages.py`, `test_hiring.py`
2.1 Кандидат: профиль вручную или из PDF с авторасознаванием; стандартизированный PDF-профиль	Форма профиля и загрузка резюме (PDF, DOCX) с разбором полей; выгрузка профиля в PDF	сайт `/candidate/profile`; API `/api/me/resume`, `/api/me/profile.pdf`; тесты `test_proctor.py` (`test_a6_pdf_resume_hidden_text_and_injection_e2e`), `test_day2_rules.py` (`test_profile_pdf`)
2.1 Кандидат: опрос, выбор грейда, тест → категория	Опрос по направлению и стеку, выбор грейда, адаптивный тест, вердикт из трёх исходов	сайт `/candidate/assessment`; API `/api/assessment/sessions`; тест `e2e/tests/01-candidate-test.spec.ts`; док. 3.2
2.1 Кандидат: категория, история опроса и теста, тест на грейд ниже или выше	Экран категории, история попыток, запуск теста на другой грейд с учётом ограничений	сайт `/candidate/category`, `/candidate/attempts`; тест `e2e/tests/15-grade-above.spec.ts`
2.1 Кандидат: приглашения с вакансией и зарплатой, принять или отклонить	Входящие приглашения с компанией, описанием и вилкой; кнопки «Принять», «Отклонить»	сайт `/candidate/invitations`; API `/api/candidate/invitations/{inv_id}/accept`, `/decline`
2.1 Кандидат (дополнительно): вакансии и отклик, статусы откликов	Список опубликованных вакансий, отклик, список своих откликов со статусами	сайт `/candidate/vacancies`; API `/api/vacancies/{vid}/apply`, `/api/candidate/applications`; тест `test_integrations.py` (`test_vacancy_flow_contacts_after_apply`)
2.1 Кандидат: настройки приватности, согласия, связь с ФСП ID	Выбор полей, видимых работодателю; согласия с отзывом; выгрузка и удаление своих данных	сайт `/candidate/settings`, `/candidate/mydata`; API `/api/me/privacy`, `/api/me/consents`; тест `test_mydata.py`
2.1 Кандидат видит только свои приглашения и отклики	Проверка владельца на каждом методе; чужая запись — ошибка 404	тесты `test_access_matrix.py`, `test_hiring.py` (`test_foreign_invitation_is_404_and_roles`), `test_mydata.py` (`test_export_contains_own_data_and_nothing_foreign`)
2.1 Работодатель: профиль компании	Описание, направление деятельности, контакты	сайт `/employer/company`; API `/api/me/company`
2.1 Работодатель: описание потребности → подборка	Текст своими словами или файл вакансии; требования выделяются с цитатами из текста	сайт `/employer/needs`; API `/api/employer/needs`; тест `e2e/tests/04-need-art64-compare.spec.ts`; док. 4.2

Пункт ТЗ	Как сделано	Где проверить
2.1 Работодатель: подборка с обоснованием	Рекомендованные категории и кандидаты, у каждого — «почему в подборке» полными фразами	сайт `/employer/needs/<id>`; код `backend/app/modules/matching/ranking.py`; док. 4.3
2.1 Работодатель: поиск по базе с фильтрами, в том числе по достижениям ФСП	Фильтры: направление, грейд, категория, стек, подтверждённые достижения ФСП	сайт `/employer/search`; API `GET /api/employer/candidates` (параметры `direction`, `grade`, `category`, `stack`, `fsp`)
2.1 Работодатель: приглашение без привязки к вакансии, статусы	Приглашение конкретному кандидату с описанием и вилкой; статусы «отправлено — просмотрено — принято — отклонено», а также «отозвано» и «истекло»	сайт `/employer/invitations`; API `POST /api/employer/invitations`; тест `test_a2_cabinets.py` (`test_expert_checklist_7_salary_before_consent_contacts_hidden_and_revoke`)
2.1 Работодатель (дополнительно): публикация вакансий и работа с откликами	Создание, редактирование, публикация и закрытие вакансии; отклики со статусами	сайт `/employer/vacancies`, `/employer/applications`; API `/api/employer/vacancies/{vid}/publish`, `/api/employer/vacancies/{vid}/applications`
2.1 Работодатель не видит контакты до принятия приглашения или отклика	Контакты закрыты до согласия; при отзыве согласия закрываются снова	тесты `test_a2_cabinets.py` (`test_expert_checklist_7_salary_before_consent_contacts_hidden_and_revoke`), `test_mydata.py` (`test_export_contacts_open_only_after_accept`)
2.1 Подбор: выдача по категориям из теста, а не из резюме	Грейд в подборке и фильтре — только подтверждённый тестом; заявленный навык не даёт баллов	тест `test_a5_expert_rule.py`; док. 4.3
2.1 Подбор: внутри категории выше — сильнее тест и подтверждённые достижения ФСП; объяснимость	Порядок: тест по навыкам вакансии, подтверждённые факты ФСП; строка «выше следующего за счёт …»	сайт `/employer/needs/<id>`; `docs/matching/ORDER.md`; тест `test_status.py` (`test_tier_order_in_ranking`)
2.1 Подбор: уточнение запроса без потери прежней подборки	Каждое уточнение создаёт новую версию подборки, прежние доступны	сайт `/employer/needs/<id>/v/<версия>`; API `POST /api/employer/needs/{need_id}/selections/{version}/refine`; тест `test_mutual.py` (`test_feedback_refine_keeps_previous_version`)
2.2 (1) Вилка «от–до» в рублях обязательна в вакансии и приглашении; кандидат видит её до общения	Ограничение в схеме API и в базе данных; приглашение без вилки не создаётся	API `POST /api/employer/invitations` без вилки — ошибка 422; тест `test_a2_cabinets.py` (`test_expert_checklist_7_salary_before_consent_contacts_hidden_and_revoke`)
2.2 (2) Приглашение: описание, вилка, компания, способ связи; контакты — после согласия	Все четыре поля обязательны; контакты кандидата раскрываются после принятия или отклика	сайт `/candidate/invitations`; тест `test_mydata.py` (`test_export_contacts_open_only_after_accept`)

Пункт ТЗ	Как сделано	Где проверить
2.2 (3) Статусы приглашений и откликов видны обеим сторонам	Один источник статуса для обеих сторон	сайт `/employer/invitations` и `/candidate/invitations`; тест `test_subs.py` (`test_expired_invitation_status_and_employer_digest`)
2.2 (4) Регистрация по почте с подтверждением адреса	Код из письма; вход до подтверждения запрещён; почта — с проверкой сертификата сервера	сайт `/register`; тесты `test_auth.py` (`test_full_flow_register_verify_login_me`, `test_login_before_verification_forbidden`), `test_mail_tls.py`
3.1 Обычная машина или недорогой сервер	Лимиты памяти контейнеров заданы в `docker-compose.yml`; языковая модель не обязательна (`LLM_DISABLED=true`)	`docker-compose.yml`; док. 9
3.2 Машинное обучение и обработка языка для резюме, категоризации и подбора	Разбор резюме и потребности, проверка развёрнутых ответов; при недоступности модели — правила и ручная проверка	`backend/app/modules/resume/`, `backend/app/modules/matching/`; `docs/validation/HH_PARSE.md`; док. 3.4, 4.2
3.3 Ссылка на развёрнутое решение, сквозной сценарий, показ механик	Рабочий сайт и раздел для экспертов; сквозные тесты повторяют сценарий из ТЗ	https://fsp-draft.ru, `/demo`; тесты `e2e/tests/01-candidate-test.spec.ts`, `e2e/tests/02-employer-invite.spec.ts`; сценарий видео `docs/video/SCRIPT.md`
3.4 Документация: архитектура; механика тестирования; подбор и категоризация; валидация; интеграция с ФСП; API; библиотеки с версиями; сборка и запуск	Восемь обязательных тем — восемь разделов	док. 2, 3, 4, 5, 6, 7, 8, 9
3.5 Открытый исходный код без компиляции и обфускации	Лицензия MIT; сторонние компоненты и их лицензии перечислены	`LICENSE`, `THIRD_PARTY.md`
3.5 Разделение на Frontend, Backend и хранение	Веб-приложение, API, PostgreSQL; модули API слабо связаны	`frontend/`, `backend/app/modules/`; док. 2
3.5 Контейнеризация	Dockerfile для каждого компонента и `docker-compose.yml`; запуск одной командой	`make up`; док. 9
3.5 API описан в OpenAPI: запросы, ответы, коды ошибок	Схема собирается из кода; тест сверяет фактические ошибки с описанием	https://fsp-draft.ru/api/docs, `docs/api/openapi.json`; тесты `test_openapi_runtime.py`, `test_openapi_models.py`; док. 7
3.5 Безопасность и 152-ФЗ: согласия, роли, защита доступа, хранение паролей и токенов	Согласия с отзывом, матрица доступа по ролям, пароли — Argon2, двухфакторный вход, секреты вне репозитория	тесты `test_access_matrix.py`, `test_auth.py` (`test_password_stored_as_argon2`), `test_mydata.py`; `gitleaks detect`; док. 10
3.5 Аутентификация, совместимая с Keycloak	Вход через «ФСП ID» по стандартному протоколу; демо-стенд на Keycloak	папка `keycloak/`; тест `test_fsp_0710.py`; док. 6

Пункт ТЗ	Как сделано	Где проверить
3.5 Кроссплатформенность и адаптивная вёрстка	Веб для десктопа, планшета и телефона; устанавливаемое приложение	сайт `/install`; тест `e2e/tests/08-mobile-390.spec.ts`
3.5 Производительность при десятках одновременных пользователей	Нагрузочный сценарий по основным операциям: подборка, профиль, поиск	`tools/loadtest/`; док. 5
3.5 Качество кода: комментарии, соответствие документации, сборка и запуск	Тесты API и интерфейса; документация и цифры собираются из кода одной командой	`make test`; `python3 docs/submission/build.py`
5.2 Репозиторий, презентация, прототип, документация в doc или pdf	Всё в репозитории и на сайте	`README.md`; `docs/presentation/`; https://fsp-draft.ru; `docs/submission/DOCUMENTATION.pdf`, `DOCUMENTATION.docx`
6 Критерии оценки 1–7 → что показываем → чем доказано	Отдельная страница с картой критериев и ссылками на экраны и проверки	сайт `/criteria`; `docs/CRITERIA.md`

Приложение Б. Все цифры документа

Каждая строка — ключ в `docs/NUMBERS.json`, значение и условие измерения. Файл собран 21.10.2026; общая команда пересчёта — `python3 tools/publish/gen_numbers.py`.

Что измерено	Значение	Условие
Верный итог проверки заявленного грейда · `grade_verdict.st.view.verdict_accuracy`	84,8 % (2 544 из 3 000); 95 % ДИ 83,5–86,0 %	[1]
Кандидатов получают уверенный ответ уже по основному тесту · `grade_verdict.st.view.confident_share`	89,2 % (2 676 из 3 000); 95 % ДИ 88,0–90,3 %	[1]
Точность уверенного ответа · `grade_verdict.st.view.confident_accuracy`	89,8 % (2 403 из 2 676); 95 % ДИ 88,6–90,9 %	[1]
Кандидатов направляется на короткий дополнительный этап · `grade_verdict.st.view.followup_share`	10,8 % (324 из 3 000); 95 % ДИ 9,7–12,0 %	[1]
Грейд определён точно · `grade_verdict.st.view.exact_grade`	77,6 % (2 329 из 3 000); 95 % ДИ 76,1–79,1 %	[1]
Ложных подтверждений от всех кандидатов · `grade_verdict.st.view.false_confirm_all`	2,1 % (63 из 3 000); 95 % ДИ 1,6–2,7 %	[1]
Ошибок больше чем на одну ступень · `grade_verdict.st.view.gross_error`	0,1 % (4 из 3 000); 95 % ДИ 0,1–0,3 %	[1]
Совпадение итога при повторном прохождении · `grade_verdict.main.retest.verdict_agreement`	72,9 % (2 187 из 3 000); 95 % ДИ 71,3–74,5 %	[2]
Совпадение грейда при повторном прохождении · `grade_verdict.main.retest.exact_agreement`	68,5 % (2 056 из 3 000); 95 % ДИ 66,8–70,2 %	[2]
Верный итог при неточно известной трудности заданий · `grade_verdict.misspec.verdict_accuracy`	84,4 % (1 688 из 2 000); 95 % ДИ 82,7–85,9 %	[3]
Верный итог при ошибках проверки развёрнутых ответов · `grade_verdict.rater_noise.verdict_accuracy`	86,1 % (1 292 из 1 500); 95 % ДИ 84,3–87,8 %	[4]

Что измерено	Значение	Условие
Среднее время теста · `grade_verdict.st.view.minutes_total.mean`	43,2 мин	[1]
Время теста у 90 % кандидатов не больше · `grade_verdict.st.view.minutes_total.p90`	69,4 мин	[1]
Семейств заданий в банке · `bank.families`	1 646	[5]
Семейств с кодом и развёрнутым ответом · `bank.code_and_open_families`	416	[5]
Направлений · `bank.directions`	10	[5]
Уникальных вариантов на проверке · `bank.unique_variants_20seeds`	32 896	[5]
Автоматических проверок ответов · `bank.checks`	38 131	[5]
Проверок с ошибкой · `bank.checks_failed`	0	[5]
Доля подходящих в первой пятёрке подборки (P@5) · `matching.p5.mean`	0,77; 95 % ДИ 0,74–0,80	[6]
MRR — насколько высоко первый подходящий кандидат · `matching.mrr.mean`	0,86	[6]
Верно определено направление вакансии · `hh_match.direction_ok`	85,1 % (286 из 336)	[7]
Атак перехвачено фильтром до оценки моделью · `anticheat.holdout_offline.rate`	77,8 % (137 из 176)	[8]
Ложных срабатываний на честных ответах · `anticheat.holdout_offline.false_positive_rate`	1,1 % (2 из 176)	[8]
Атак прошли всю цепочку защиты · `anticheat.llm_chain.pierced`	0 из 176	[9]
Честных кандидатов ошибочно отправлено на проверку · `anticheat.behavioral_followup.honest_false_followup`	0 из 1 000	[10]
Модельных ботов отправлено на дополнительную проверку · `anticheat.behavioral_followup.bot_detection_rate`	100 % (500 из 500)	[10]

Что измерено	Значение	Условие
Официальных протоколов в индексе · `fsp_protocols.index.documents`	18	[11]
Соревнований · `fsp_protocols.index.events`	27	[11]
Результатов · `fsp_protocols.index.results`	2 908	[11]
Спортсменов · `fsp_protocols.index.athletes`	2 232	[11]
Субъектов РФ · `fsp_protocols.index.regions`	69	[11]
Спортсменов с выполненным нормативом ЕВСК · `fsp_protocols.index.athletes_with_evsk_norm`	130	[11]

1. Стандартные условия (как в аналогичных решениях), зёрна 5501–5510; симуляция продукта: модельные кандидаты, не люди; N = 3 000; тот же движок теста, что на сайте; банк ab16d27 (1 646 семейств заданий); 21.10.2026. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase st --n 300 --seeds 5501 --out /w/a4fin2/out/b/S_T__5501.json --population st --world st --bank-a mcq=1.5,code=1.5 --retest --fixed --expect-bank-hash 913ad19f6a259aff`.
2. Стандартные условия (как в аналогичных решениях), зёрна 5501–5510; симуляция продукта: модельные кандидаты, не люди; N = 3 000; тот же движок теста, что на сайте; банк ab16d27 (1 646 семейств заданий); 21.10.2026; повтор на другом наборе заданий. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase st --n 300 --seeds 5501 --out /w/a4fin2/out/b/S_T__5501.json --population st --world st --bank-a mcq=1.5,code=1.5 --retest --fixed --expect-bank-hash 913ad19f6a259aff`.
3. Симуляция продукта, модельные кандидаты; N = 2 000; трудность и различающая сила заданий в «мире» сдвинуты случайно, движок об этом не знает; 21.10.2026. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase final --n 200 --seeds 4401 --out /w/a4fin2/out/main/F_misspec__4401.json --world misspec --expect-bank-hash 913ad19f6a259aff`.
4. Симуляция продукта, модельные кандидаты; N = 1 500; проверка развёрнутых ответов ошибается случайно; 21.10.2026. Пересчёт: `cd backend && python -m app.modules.validation.product_sim --phase final --n 150 --seeds 4401 --out /w/a4fin2/out/main/F_rater__4401.json --rater-noise 0.1 --expect-bank-hash 913ad19f6a259aff`.
5. Автоматическая проверка банка: 20 вариантов-зёрен на каждое семейство, коммит банка ab16d27, 21.10.2026. Пересчёт: `cd backend && python -m assessment_bank validate --seeds 20`.
6. Модель: 210 потребностей × 900 модельных кандидатов на отложенных мирах (истина известна, не люди); миры заморожены до прогона, веса подбирались на других мирах; 21.10.2026. Пересчёт: `python3 tools/realdata/bench_holdout4.py run (один раз после freeze; повтор запрещён) → python3 tools/realdata/numbers_matching.py`.
7. Реальные вакансии hh.ru: второй отложенный набор: 336 свежих вакансий hh.ru (SHA-256 2bb77a1cd8d7…, заморожен 2026-10-21, прогон один); метки — поля hh.ru (роль и слова названия), разбор текста правилами без LLM; прогон 2026-10-21, код 249443367; N = 336; 21.10.2026. Пересчёт: `cd backend && python ../tools/realdata/hh_holdout.py --tag 2 run (один раз; новый прогон — новый отложенный набор: collect → freeze → run)`.

8. Отложенный набор атак на проверку ответов, составлен после фиксации кода фильтра (коммит 6b7c710c), один прогон без донастройки; N = 176 атак и 176 честных ответов; 20.10.2026. Пересчёт: `python3 tools/grader_check/run_holdout_eval.py --offline`.
9. Отложенный набор атак на проверку ответов, составлен после фиксации кода фильтра (коммит 6b7c710c), один прогон без донастройки; N = 176 атак и 176 честных ответов; 20.10.2026; полная цепочка проверки. Пересчёт: `python3 tools/grader_check/run_holdout_eval.py --llm --max-cost-usd 3.0`.
10. Симуляция поведения: 1 000 модельных честных сессий и 500 модельных ботов; правило «два и больше сигнала или прямое доказательство», автоматической блокировки нет. Пересчёт: `python3 tools/grader_check/run_holdout_eval.py --offline`.
11. Официальные протоколы ФСП России 2025–2026 с сайта федерации; судьи, тренеры и справки исключены; 21.10.2026. Пересчёт: `python3 tools/fsp_protocols/export_public.py /opt/spectrek_fsp/data_local/fsp_protocols/index_v7.json docs/realdata/fsp_protocols backend/app/modules/fsp`.